

TutorFlow: a session platform for online tutors

Build a small web app with two types of users. Tutors manage their students, run sessions, and use AI to prepare and review each session. Students log in to see their own sessions and homework.

Full stack

Two user roles

AI integration

Database

EdTech

7 days

7
DAYS

You have 7 days from the day you receive this task. Send a live working URL and a GitHub repository link. Put one tutor login and one student login in the README so we can test the app without setting anything up.

WHAT YOU ARE BUILDING

TutorFlow is a platform for online tutors who teach students one to one. A tutor can add students, schedule sessions, take notes during a session, and use AI to plan the session before it starts and summarise it afterwards. A student can log in and see their upcoming sessions, their past session notes, and the homework the AI generated for them.

HOW TO APPROACH IT

There is no single correct answer here. Pick a stack you already know, keep the app simple, and make sure the core flows work end to end. We would rather see six features that work properly than eight that half work. Explain your choices in the README.

IF YOU CANNOT FINISH EVERYTHING

Submit what you have. This is expected and it is not a penalty. A partly built app with an honest account of what is missing is worth more to us than a link that does not open, and far more than nothing at all.

If some features are incomplete, add a short section at the top of your README listing what works, what does not, and what you would have done next. We will read your submission against what you say it does.

THE ONE THING THAT MUST WORK

Whatever you submit has to open and run from the link. A tutor should be able to log in and reach at least one working screen. Everything beyond that is scored on what you built, not on what is missing.

THE SESSION LIFECYCLE



Every session moves through these four states in order. Your code should not allow a session to skip a state. For example, a session cannot go straight from Scheduled to AI reviewed. Once a session is Completed, nothing can be edited except triggering the AI review.

TOOLS YOU NEED TO USE

Two external services are required:

An AI model. OpenAI, Claude, or Gemini. Free tiers are fine.

A database. Postgres, MongoDB, SQLite, or Supabase.

WRITE THIS IN THE README

Describe your database tables and how they relate to each other. Also paste the prompts you send to the AI and explain why you wrote them that way. Prompt quality is scored. A prompt that only says "generate a lesson plan" will score low, because it ignores everything you know about the student.

A SUGGESTED ORDER OF WORK

You have a week. If you are not sure where to start, this order keeps you unblocked:

- Deploy an empty page on day one, so deployment is never what breaks at the end.
- Build login and the two roles next. Everything else sits on top of it.
- Design your database tables before you write screens. Changing tables later is the slowest kind of rework.
- Get scheduling and notes working with plain data, then add the AI calls.
- Leave the last day for error states, loading states, test accounts, and the README.

CHOOSING YOUR STACK

Use whatever you can build and deploy fastest. These combinations are easy to host for free:

Next.js and Supabase

React with Node, Express and Postgres

Next.js and MongoDB Atlas

Any stack you already know well

IMPORTANT

We must be able to open your link and use the app straight away. Nothing should need to run on our machine. Put all API keys and environment variables on the deployment, and put the test logins in the README.

FEATURES TO BUILD

1 Login with two roles

Tutor accounts and student accounts. Use JWT or sessions, whichever you know. A tutor sees only their own students and sessions. A student sees only their own data. There is no public sign up. Tutors create student accounts.

2 Student profile

Each student has a name, subject, current level, learning goals, and weak areas. Weak areas can be free text. This profile is what the AI reads, so it must be available when you build the AI features.

3 Scheduling a session

The tutor picks a student, a date and time, and a topic. One tutor cannot have two sessions at the same time, so check for a clash before saving. Add basic form validation.

4 AI session plan

Before a session starts, the tutor clicks a button and the AI returns a plan: learning objectives, a lesson outline of four points, and three practice questions. Send the student profile and past sessions to the AI so the plan fits that student.

5 Session notes with autosave

While a session is in progress, the tutor types notes and they save on their own. If the tutor closes the tab and comes back, the notes are still there. Use a debounce so you are not saving on every keystroke.

6 AI session review

After a session is completed, the tutor clicks a button and the AI reads the notes. It returns a summary of the session, two or three homework tasks for the student, and one suggestion for what to cover next time.

7 Progress view

The tutor opens a student and sees all their sessions in order. A progress summary button sends every past AI review to the AI and returns a short paragraph on where the student is improving and where they still struggle.

8 Student view

A student logs in and sees three things: upcoming sessions, notes from completed sessions in read only form, and their homework. Keep this screen simple.

BONUS, NOT REQUIRED

Email the student when a tutor schedules a session. Resend, SendGrid, or any free mailer works.

HOW YOUR WORK IS SCORED, OUT OF 100

Login and access	Roles are checked on the server, not only in the interface. A student cannot open a tutor page. One tutor cannot see another tutor's data.	15
Session states	States move in the correct order, no session can be double booked, and the state shown on screen matches the state in the database.	15
AI features	Prompts use the student profile and past sessions. The output is structured and useful. The app does not break when the AI call fails.	25
Database and backend	Sensible tables and relationships, no repeated queries inside loops, and autosave that works with a debounce.	20
Frontend	Loading states, clear error messages, and both the tutor and student screens working and easy to move around.	15
README	Database structure described, prompts explained, and an honest note on what you would improve.	10
Total		100

WHAT TO SUBMIT

Live URL

Deployed, reachable, and loaded with some test data.

GitHub repository

Public, or shared with the reviewer.

README

Database structure and your AI prompts.

Test logins

One tutor account and one student account.

ONE LAST THING

Add five sentences at the end of your README on what you would build next if you had another day. This tells us how you think about the product, not just the code.

QUESTIONS

If anything here is unclear, email us and ask.